

Softmax, Properly

From any scores to probabilities · video 1:52

This lesson has one big idea: **softmax turns any list of scores into probabilities**, in two steps. Exponentiate every score, then divide by the total. Everything else follows from those two steps: why it is “soft”, why only the differences between scores matter, how temperature sharpens or flattens the result, and why code subtracts the largest score first.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Scores in, probabilities out	▶ 0:08–0:36	4	☐
2. Working it through: 3, 2, 0	▶ 0:36–0:50	4	☐
3. A soft version of the max	▶ 0:51–1:01	4	☐
4. Only the differences matter	▶ 1:02–1:08	4	☐
5. Temperature: sharper or flatter	▶ 1:09–1:19	4	☐
6. Subtract the max: softmax in code	▶ 1:20–1:40	3	☐

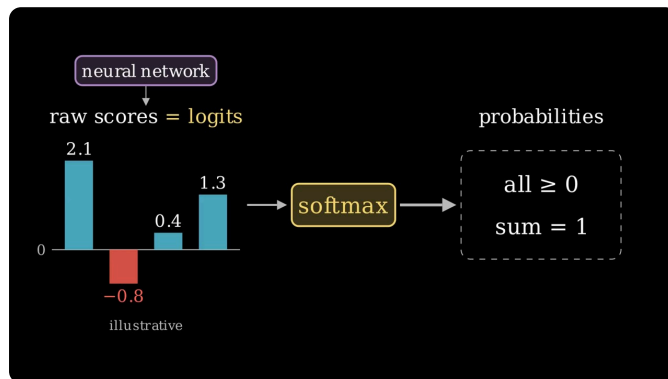
Before you start: this lesson builds on F05 (Exponentials and Logarithms): e^x is always positive and exaggerates differences; and on F06 (Probability and Sampling): a distribution is a list of probabilities that adds up to 1. A calculator helps for the Checks. Softmax is used in episodes 5 and 6 (attention scores become weights) and in episode 10 (logits become word probabilities).

CONCEPT 1 OF 6

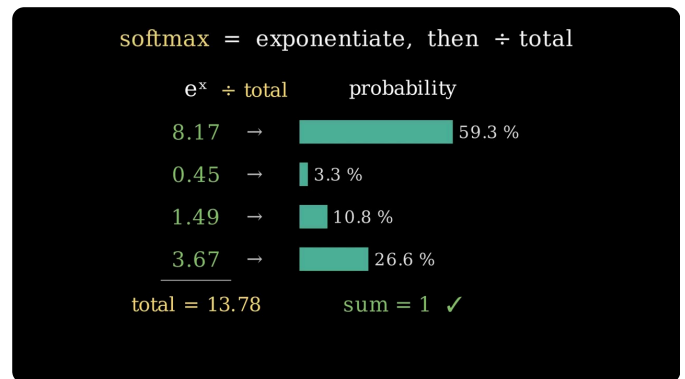
Scores in, probabilities out

▶ Video 0:08–0:36

A neural network ends with raw scores called **logits**, one per option. They can be any number: large, small or negative (the video uses 2.1, −0.8, 0.4 and 1.3). But probabilities must be ≥ 0 and **add up to 1**. **Softmax** converts in two steps. **Step 1: exponentiate** every score; e^x is always positive, so even −0.8 becomes 0.45. **Step 2: divide** each result by their total (13.78). Now they add up to exactly 1.



▶ 0:20 Logits can be any number; softmax makes them probabilities.



▶ 0:35 The recipe on the illustrative logits (total 13.78).

KEY IDEA

softmax = **exponentiate, then divide by the total**. Probability of option $i = e^{\text{score } i} \div (\text{sum of all the } e^{\text{score}})$. Step 1 makes every value positive; step 2 makes them add up to 1.

Check 1: Scores in, probabilities out

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Why not skip step 1, and simply divide the raw scores by their total?

- A. It would give the same result, only more slowly
- B. Raw scores can be negative, so some results would be negative
- C. Division only works on whole numbers
- D. The results would add up to more than 1

1.2 TRUE OR FALSE

“Logits must be between 0 and 1, just like probabilities.”

True ☐ False ☐

1.3 CALCULATION

Apply softmax by hand to the two scores **1** and **0**. (Use $e \approx 2.718$.)

1.4 PUT IN ORDER

Put in order: *divide by the total · raw scores (logits) · probabilities · exponentiate*.

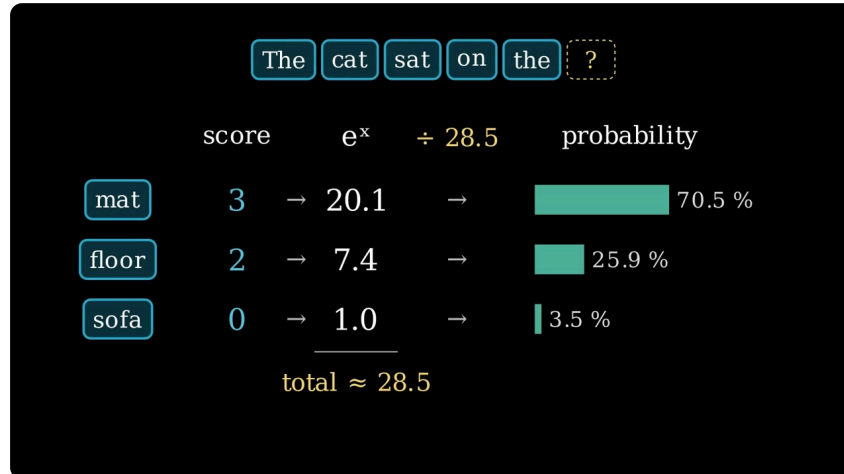
CONCEPT 2 OF 6

Working it through: 3, 2, 0

▶ Video 0:36–0:50

The video's example: after “*The cat sat on the*”, the scores for *mat*, *floor* and *sofa* are 3, 2 and 0.

Exponentiate: $e^3 \approx 20.1$, $e^2 \approx 7.4$, $e^0 = 1.0$. **Add:** the total is about 28.5. **Divide:** $20.1 \div 28.5 \approx 70.5\%$, $7.4 \div 28.5 \approx 25.9\%$, $1.0 \div 28.5 \approx 3.5\%$.



▶ 0:50 The video's example: scores 3, 2, 0 become 70.5 %, 25.9 % and 3.5 %.

Two things to notice. The scores 3 and 2 differ by only 1, yet *mat* gets almost three times the probability of *floor*: e^x exaggerates differences. And a score of 0 is not “no chance”: $e^0 = 1$, so *sofa* still gets 3.5 %.

KEY IDEA

By hand: **exponentiate each score, add them up, divide each one by the total**. Check your work: the results must add up to 100 % ($70.5 + 25.9 + 3.5 = 99.9$, off only by rounding).

Check 2: Working it through: 3, 2, 0

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Four words all get the same score, 2. What probability does each one get?

2.2 CALCULATION

Compute the softmax of the scores **1, 0, 0**. (Use $e \approx 2.718$.)

2.3 CALCULATION

Drop *sofa* and keep only *mat* (score 3) and *floor* (score 2). Using $e^3 \approx 20.1$ and $e^2 \approx 7.4$, what are their new probabilities?

2.4 TRUE OR FALSE

“A word with a score of 0 gets a probability of 0.”

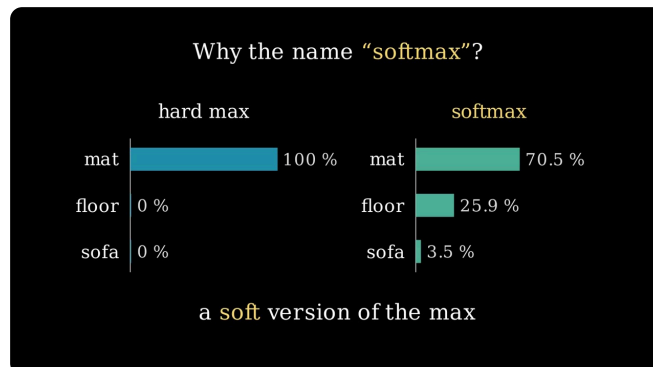
True ☐ False ☐

CONCEPT 3 OF 6

A soft version of the max

▶ Video 0:51-1:01

A **hard max** gives all the probability to the top score: *mat* 100 %, *floor* 0 %, *sofa* 0 %. Softmax **leans toward** the top score (70.5 %) but still gives the others a share: a soft version of the max.



▶ 1:01 Hard max: all to the top score. Softmax: a share for everyone.

The order never changes: a higher score always gets a higher probability, because e^x always grows. The bigger the gaps between the scores, the closer softmax gets to a hard max.

KEY IDEA

Hard max: the winner takes everything. Softmax: the winner takes the most, and everyone else keeps a share, **in the same order as the scores**.

Check 3: A soft version of the max

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

For the illustrative logits 2.1, -0.8, 0.4, 1.3, what does a **hard max** output?

- A. 100 %, 0 %, 0 %, 0 %
- B. 59.3 %, 3.3 %, 10.8 %, 26.6 %
- C. 25 %, 25 %, 25 %, 25 %
- D. 0 %, 100 %, 0 %, 0 %

3.2 CALCULATION

Compute the softmax of the scores **4** and **0** ($e^4 \approx 54.6$). Is it closer to a hard max than 3, 2, 0 was?

3.3 TRUE OR FALSE

"Softmax can give a lower score a higher probability than a higher score."

True ☐ False ☐

3.4 MULTIPLE CHOICE

Which statement is **always** true for softmax?

- A. The top score gets more than 50 %
- B. Every option gets more than 0 %, and higher scores get higher probabilities
- C. The lowest score gets 0 %
- D. Probabilities are proportional to the scores

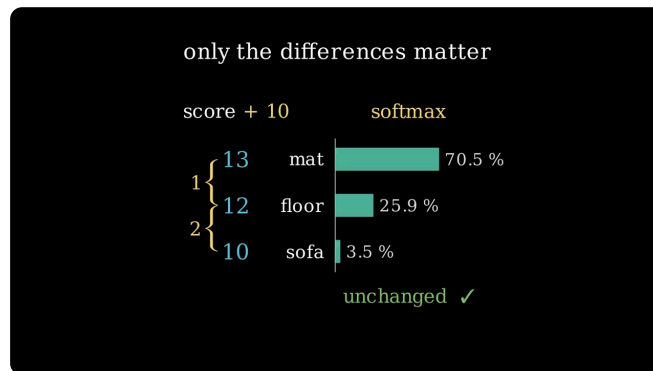
CONCEPT 4 OF 6

Only the differences matter

▶ Video 1:02-1:08

Add 10 to every score: 3, 2, 0 become 13, 12, 10. The probabilities do not change at all: still 70.5 %, 25.9 %, 3.5 %. The gaps between the scores (1 and 2) are the same, and **only the gaps matter**.

Why: $e^{x+c} = e^x \times e^c$, so adding c multiplies every value, and the total, by the same factor, which cancels when you divide. So a gap of d always makes one option **e^d times as likely** as the other: *mat* vs *floor* (gap 1) is $e \approx 2.72$ times ($70.5 \div 25.9$).



▶ 1:08 Add 10 to every score: the gaps stay 1 and 2, so nothing changes.

KEY IDEA

Adding the same number to every score changes nothing. Only the differences matter: a gap of d makes one option e^d times as likely as the other.

Check 4: Only the differences matter

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Compute the softmax of the scores **-1, -2, -4**. (No new calculation needed.)

4.2 MULTIPLE CHOICE

The scores 1 and 0 give 73.1 % and 26.9 %. Which scores give exactly the same result?

- A. 2 and 0
- B. 101 and 100
- C. 10 and 0
- D. 0.5 and 0

4.3 CALCULATION

In the video's example, how many times as likely is *mat* (score 3) as *sofa* (score 0)? Use the gap, then check with the percentages.

4.4 TRUE OR FALSE

"Multiplying every score by 2 also leaves the probabilities unchanged."

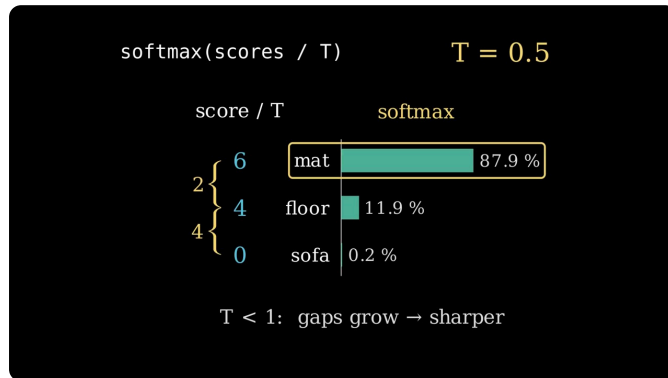
True ☐ False ☐

CONCEPT 5 OF 6

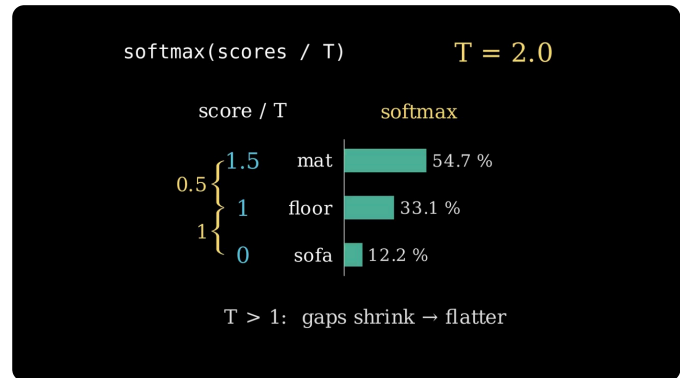
Temperature: sharper or flatter

▶ Video 1:09-1:19

Before the softmax, we can divide the scores by a **temperature** T : $\text{softmax}(\text{scores} / T)$. With $T = 0.5$, the scores 3, 2, 0 become 6, 4, 0: the gaps double and the top choice dominates (87.9 %, 11.9 %, 0.2 %). With $T = 2$, they become 1.5, 1, 0: the gaps halve and the distribution flattens (54.7 %, 33.1 %, 12.2 %).



▶ 1:16 $T = 0.5$: gaps double, mat dominates.



▶ 1:18 $T = 2$: gaps halve, the distribution flattens.

KEY IDEA

Divide the scores by T , then apply softmax. **$T < 1$: the gaps grow → sharper** (toward a hard max). **$T > 1$: the gaps shrink → flatter** (toward equal shares). $T = 1$ is plain softmax.

Check 5: Temperature: sharper or flatter

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

Scores 3, 2, 0 with $T = 0.25$. (a) What are the scores after dividing by T ? (b) What are the gaps? (c) Will *mat* get more or less than the 87.9 % it gets at $T = 0.5$?

5.2 PUT IN ORDER

Scores 3, 2, 0. Order these temperatures by the probability *mat* gets, lowest first: $T = 0.5 \cdot T = 10 \cdot T = 1 \cdot T = 2$.

5.3 TRUE OR FALSE

“Dividing the scores by $T = 0.5$ gives the same probabilities as multiplying every score by 2.”

True ☐ False ☐

5.4 MULTIPLE CHOICE

A chatbot keeps giving the same safe, predictable answer. You want more varied word choices. What should you change?

- A. Lower the temperature to 0.3
- B. Raise the temperature to 1.3
- C. Add 5 to every score
- D. Subtract the largest score

CONCEPT 6 OF 6

Subtract the max: softmax in code

▶ Video 1:20-1:40

A computer cannot store e^{1000} : the largest 64-bit float is about $e^{709} \approx 8.2 \times 10^{307}$, so e^{1000} **overflows** to ∞ . The fix: **subtract the largest score first**. For 3, 2, 0 that gives 0, -1, -3; $e^x \approx 1.00, 0.37, 0.05$; total ≈ 1.42 ; divided out: 70.5 %, 25.9 %, 3.5 %. Identical, because only differences matter.

A practical trick: **subtract the max**

$e^{1000} = \infty$ (overflow)

largest float64 $\approx e^{709} \approx 8.2 \times 10^{307}$

$[3, 2, 0] - 3 = [0, -1, -3]$ - the largest score

$e^x \approx [1.00, 0.37, 0.05]$ total ≈ 1.42

$\div$ total $\rightarrow [70.5 \%, 25.9 \%, 3.5 \%]$ identical ✓

✓ no overflow

▶ 1:32 Subtract the largest score: identical answer, no overflow.

In code that is three lines: divide by the temperature, exponentiate after subtracting the max, and normalize.

```
def softmax(scores, temperature=1.0):
    z = np.array(scores) / temperature
    e = np.exp(z - z.max())      # no overflow
    return e / e.sum()

softmax([3, 2, 0])              # [0.705, 0.259, 0.035]
softmax([3, 2, 0], 0.5)        # [0.879, 0.119, 0.002]
```

KEY IDEA

Subtracting the max changes nothing mathematically (concept 4), but every exponent becomes ≤ 0 , so every e^x lies between 0 and 1 and **nothing overflows**.

Check 6: Subtract the max: softmax in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

Scores 1000, 999, 997. (a) What are they after subtracting the max? (b) What are the probabilities?

6.2 TRUE OR FALSE

"After subtracting the max, the top score always becomes $e^0 = 1$, and every other value lies between 0 and 1."

True ☐ False ☐

6.3 HANDS-ON CODE

Try it yourself. Below are the video's `softmax` and a `naive_softmax` that skips the subtract-the-max step. Predict each output first, then run it. (a) `naive_softmax([1000, 999, 997])`. Why? (b) `softmax([1000, 999, 997])`. (c) `softmax([3, 2, 0], 0.01)` and `softmax([3, 2, 0], 100)`: which one looks like a hard max?

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)

def naive_softmax(scores):
    e = np.exp(np.array(scores, dtype=float))
    return e / e.sum()

def softmax(scores, temperature=1.0):
    z = np.array(scores) / temperature
    e = np.exp(z - z.max())      # no overflow
    return e / e.sum()

print(softmax([3, 2, 0]))      # [0.705 0.259 0.035]
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Scores in, probabilities out • rewatch ▶ 0:08-0:36

1.1 B. For 2.1, -0.8, 0.4, 1.3 the total is 3.0, and $-0.8 \div 3.0 \approx -0.27$: not a probability. e^x makes every value positive first.

1.2 False. Logits can be any number, even negative. Only the output of softmax has to be ≥ 0 and add up to 1.

1.3 73.1 % and 26.9 %. $e^1 \approx 2.718$ and $e^0 = 1$, total 3.718. Then $2.718 \div 3.718 \approx 0.731$ and $1 \div 3.718 \approx 0.269$, which add up to 1.

1.4 raw scores → exponentiate → divide by the total → probabilities. Exponentiate first (everything positive), then normalize (everything adds up to 1).

2. Working it through: 3, 2, 0 • rewatch ▶ 0:36-0:50

2.1 25 %. All four values are $e^2 \approx 7.39$, so each is $7.39 \div (4 \times 7.39) = \frac{1}{4}$. Equal scores always give equal shares, whatever the score.

2.2 57.6 %, 21.2 %, 21.2 %. The values are 2.718, 1 and 1, total 4.718. $2.718 \div 4.718 \approx 0.576$ and $1 \div 4.718 \approx 0.212$.

2.3 About 73.1 % and 26.9 %. The total is $20.1 + 7.4 = 27.5$, and $20.1 \div 27.5 \approx 0.731$. Notice: the same as the scores 1 and 0 in 1.3. Concept 4 explains why.

2.4 False. $e^0 = 1$, so the word keeps a share: *sofa* gets 3.5 %. Softmax never gives exactly 0.

3. A soft version of the max • rewatch ▶ 0:51-1:01

3.1 A. Everything goes to the top score, 2.1. B is what softmax gives.

3.2 98.2 % and 1.8 %. **Yes.** $54.6 \div 55.6 \approx 0.982$. A gap of 4 is large, so softmax is almost a hard max.

3.3 False. e^x always grows with x , so the ranking never changes.

3.4 B. A fails for equal scores (three options get 33.3 % each); C fails because $e^x > 0$; D fails: scores 3 and 2 give 70.5 % and 25.9 %, 2.7 times as much, not 1.5 times.

4. Only the differences matter • rewatch ▶ 1:02-1:08

4.1 70.5 %, 25.9 %, 3.5 %. They are 3, 2, 0 minus 4: the same gaps (1 and 2), so the same probabilities.

4.2 B. The gap is still 1. The others change the gap (2, 10, 0.5), so they change the probabilities.

4.3 About 20 times. The gap is 3, and $e^3 \approx 20.1$. Check: $70.5 \div 3.5 \approx 20$.

4.4 False. Doubling changes the gaps (1 and 2 become 2 and 4), so the result is sharper: 87.9 %, 11.9 %, 0.2 %. That is temperature, the next concept.

5. Temperature: sharper or flatter · rewatch ► 1:09-1:19

5.1 (a) 12, 8, 0. (b) 4 and 8. (c) More: 98.2 %. $T = 0.25$ multiplies the gaps by 4, even more than $T = 0.5$ does, so the result is even sharper.

5.2 $T = 10 \rightarrow T = 2 \rightarrow T = 1 \rightarrow T = 0.5$. 37.8 %, 54.7 %, 70.5 %, 87.9 %: the higher the temperature, the flatter the distribution.

5.3 True. Dividing by 0.5 is multiplying by 2. Both give 6, 4, 0 $\rightarrow$ 87.9 %, 11.9 %, 0.2 %.

5.4 B. $T > 1$ flattens the distribution, so sampling (F06) picks less likely words more often. C and D change nothing (concept 4).

6. Subtract the max: softmax in code · rewatch ► 1:20-1:40

6.1 (a) 0, -1, -3. (b) 70.5 %, 25.9 %, 3.5 %. The gaps are 1 and 2, as for 3, 2, 0, so these are exactly the video's numbers.

6.2 True. The top score becomes 0, and all the others become negative, so their e^x is below 1.

6.3 (a) [nan nan nan] · (b) [0.705 0.259 0.035] · (c) [1. 0. 0.] and [0.338 0.334 0.328] (a) `np.exp(1000.0)` overflows to `inf` (NumPy prints a warning), and `inf ÷ inf` is `nan`, “not a number”. (b) Subtracting 1000 gives 0, -1, -3, the video's example. (c) $T = 0.01$ makes the gaps 100 times bigger, so `mat` takes everything: a hard max. $T = 100$ shrinks the gaps to 0.01 and 0.02, so the shares are almost equal.